```
def sumofdigits(n):
    if n == 0:
        return 0
    return (n % 10) + sumofdigits(n // 10)
print(sumofdigits(12345))
```

```
15
```

```
#palindrome using recursion
def palindrome(m):
  if m < 10:
    return m
  return (m % 10) * 10 + palindrome(m // 10)
print(palindrome(121))
```

```
11
```

```
def palindrome(s):
  if len(s) < 1:
    return True
  if s[0] != s[-1]:
    return False
  return palindrome(s[1:-1])
print(palindrome("madam"))
```

```
True
```

```
def power(base, power):
  if power == 0:
    return 1
  return base ** power
print(power(2, 3))
```

```
8
```

```
def power(base, exponent):
    if exponent == 0:
        return 1
    return base * power(base, exponent - 1)

print(power(2, 3))
```

```
8
```

```
#flatten a nested list using recursion
#print all
```

OOPS – Class, Object, Encapsulation, Polymorphism, Inheritence, Data Abstracion

Class is the blueprint & Object is instance of a class.

Encapsulation is binding data together

Polymorphism Funcionality same, forms different

Data abstraction hides unwanted data from the user.

Inheritence gives properties/attributes from parent class to child class

```
class Student:
  pass
s1 = Student()
s2 = Student()
print(s1 == s2)
```

```
False
```

```
class Mobile:
  def __init__(self, storage, battery):
    self.storage = storage
    self.battery = battery
  def display_details(self):
    print(f"storage: {self.storage} , Battery: {self.battery}")

i11 = Mobile("6GB", "3000 mah")
i22 = Mobile(8, 5000)
i11.display_details()
i22.display_details()
```

```
storage: 6GB , Battery: 3000 mah
storage: 8 , Battery: 5000
```

```
class Students:
  def __init__(self, name, age, gender, branch):
    self.name = name
    self.age = age
    self.gender = gender
    self.branch = branch
  def display_details(self):
    print(f"Name: {self.name} , Age: {self.age} , Gender: {self.gender} , Branch: {self.branch}")

s1 = Students("Harsha", 21, "Female", "Mech")
s2 = Students("Atul", 21, "Male", "Mech")
s3 = Students("Sai", 21, "Male", "Mech")
s4 = Students("Thanos", 21, "Female", "Aerospace")
s1.display_details()
s2.display_details()
s3.display_details()
s4.display_details()

if s1.age > s2.age:
  print(f"\n{s1.name} is elder than {s2.name}")
else:
  print(f"\n{s2.name} is elder than {s1.name}")

if s3.age > s4.age:
  print(f"\n{s3.name} is elder than {s4.name}")
else:
  print(f"\n{s4.name} is elder than {s3.name}")
```

```
Name: Harsha , Age: 21 , Gender: Female , Branch: Mech
Name: Atul , Age: 21 , Gender: Male , Branch: Mech
Name: Sai , Age: 21 , Gender: Male , Branch: Mech
Name: Thanos , Age: 21 , Gender: Female , Branch: Aerospace

Atul is elder than Harsha

Thanos is elder than Sai
```

```
#stacksandQueues
```

```
class stackimplement:
  def __init__(self):
    self.stack = []
  def push(self, item):
    self.stack.append(item)
  def pop(self):
    return self.stack.pop()
  def peek(self):
    return self.stack[-1]
  def is_empty(self):
    return len(self.stack) == 0
  def size(self):
    return len(self.stack)
  def display(self):
    print(self.stack)

s = stackimplement()

s.push(1)
s.push(2)
s.push(3)
```

```
s.push(4)
s.is_empty()
s.display()
s.pop()
s.display()
s.peek()
s.display()
s.display()
```

```
[1, 2, 3, 4]
[1, 2, 3]
[1, 2, 3]
[1, 2, 3]
3
```

```
class QueueImplement:
  def __init__(self):
    self.queue = []
  def enqueue(self, item):
    self.queue.append(item)
  def dequeue(self):
    return self.queue.pop(0)
  def is_empty(self):
    return len(self.queue) == 0
  def size(self):
    return len(self.queue)
  def display(self):
    print(self.queue)

q = QueueImplement()

q.enqueue(1)
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.is_empty()
q.display()
q.dequeue()
q.display()
q.size()
```

```
[1, 2, 3, 4]
[2, 3, 4]
3
```

```
class Solution(object):
    def isValid(self, s):
        stack = []
        for ch in s:
          if ch == '(' or ch == '{' or ch == '[':
            stack.append(ch)
          else:
            if not stack:
              return False
            top = stack.pop()
            if ch == ")" and top != "(":
              return False
            if ch == "}" and top != "{":
              return False
            if ch == "]" and top != "[":
              return False
        return not stack
s = Solution()
print(s.isValid("{[}"))
```

```
False
```

```
S = ")()("
print(S[::-1])
```

```
()()
```

```
class MinStack(object):
    def __init__(self):
```

```
        self.stack = []
        self.min_stack = []

    def push(self, value):
        self.stack.append(value)
        if not self.min_stack or value <= self.min_stack[-1]:
            self.min_stack.append(value)

    def pop(self):
        if self.stack[-1] == self.min_stack[-1]:
            self.min_stack.pop()
        self.stack.pop()

    def top(self):
        return self.stack[-1]

    def getMin(self):
        return self.min_stack[-1]

    def display(self):
        print(self.stack)
        print(self.min_stack)


# Your MinStack object will be instantiated and called as such:
# obj = MinStack()
# obj.push(value)
# obj.pop()
# param_3 = obj.top()
```

```
class MyQueueusingstack(object):
    def __init__(self):
        self.stack1 = []
        self.stack2 = []

    def push(self, x):
        self.stack1.append(x)


    def pop(self):
        if not self.stack2:
            while self.stack1:
                self.stack2.append(self.stack1.pop())
        return self.stack2.pop()


    def peek(self):
        if not self.stack2:
            while self.stack1:
                self.stack2.append(self.stack1.pop())
        return self.stack2[-1]


    def empty(self):
        return not self.stack1 and not self.stack2


# Your MyQueue object will be instantiated and called as such:
# obj = MyQueue()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.peek()
# param_4 = obj.empty()
```

```
def Steps(n):
  if n < 3:
    return n
  else:
    return Steps(n-1) + Steps(n-2) + Steps(n-3)
print(Steps(5))
```

```
11
```

```
def AP(a, d, n):
  if n == 1:
```

```
    return a
  else:
    return AP(a, d, n-1) + d
```

```
4
```

```
n = int(input())

if n % 2 == 0:
  print("Divisible by 2")
elif n % 2 and n % 3 == 0:
  print("Divisible by 6")
elif n % 5 == 0:
  print("Divisible by 5")
elif n % 5 == 0 and n % 10 == 0:
  print("Divisible by 10")
elif n % n == 0:
  print("Divisible by itself")
else:
  print("None of the above")
```

```
100
Divisible by 2
```